

CricSim-X: Project Documentation

Live-Contextual Match Simulator & Strategic Franchise Optimizer for IPL 2026

1. Executive Summary & Value Proposition

In predictive sports analytics, basic models often suffer from a **"static prediction paradox"**—where global tournament strength forecasts fail to align with dynamic match-by-match fixture realities. For instance, a pipeline might evaluate a franchise like *Royal Challengers Bengaluru (RCB)* to possess the highest tournament-winning index based on historical squad depth simulations, yet fail to capture their low structural resilience under specific localized constraints (e.g., chasing slow targets at spinner-friendly venues).

CricSim-X resolves this architectural gap by implementing an interactive, end-to-end Machine Learning pipeline combined with a live contextual match-state simulation framework. Built on 18 seasons of detailed IPL ball-by-ball records (2008–2025), CricSim-X transitions prediction layers from rigid pre-match estimates into agile, real-time decision assets tailored for team strategists, broadcasters, and high-volume fantasy gaming analysis.

2. System Architecture & Modular Design

The system utilizes a decoupled design pattern to ensure scalability during parallelized Monte Carlo operations.

```
IPL-CricSim-X/
├── config.py           # Global parameters, weights, and file paths
├── main.py            # Unified orchestration CLI
├── requirements.txt    # Standard system dependencies
├── data/
│   ├── raw/           # IPL.csv (2008-2025), ipl-2026-UTC.csv
│   ├── processed/     # Scaled feature matrices and target vectors
│   └── db/             # High-performance SQLite lookup database
├── src/
│   ├── data_pipeline.py # ETL, cleaning routines, schema mapping
│   ├── feature_engineering.py # Exponential Moving Averages & venue factors
│   ├── model_factory.py  # Stacking Meta-Classifiers (XGBoost, LightGBM)
│   └── simulator.py      # Monte Carlo Markov Chain engine
└── outputs/
    ├── models/         # Serialized binaries (.pkl / .gbtree)
    └── results/        # Live simulations and analytical reports
```

3. Machine Learning Performance Metrics

To preserve predictive robustness under intense match-state variation, a Stacking Meta-Classifer blends diverse algorithmic paradigms. The following table showcases model behavior evaluated across stratified historical test sets:

Model Layer	Cross-Validation Accuracy	Test Accuracy	Test AUC-ROC
Random Forest	0.6350	0.6711	0.6995
XGBoost	0.6311	0.6534	0.7111
LightGBM	0.6477	0.6600	0.7138
Neural Network (MLP)	0.6080	0.6049	0.6141
ExtraTrees	0.6444	0.6512	0.7083
CricSim-X Meta-Ensemble	—	0.6490	0.7054

4. Advanced Feature Engineering

Raw ball-by-ball variables are mapped into multi-dimensional state indicators. The critical math pipelines include:

A. Team Form Index via Exponential Moving Averages (EMA)

Instead of treating historic performance uniformly, team strength vectors give higher relevance to recent form using an explicit decay parameter:

$$EMA_t = (NetRunRate_t \times \alpha) + (EMA_{t-1} \times (1 - \alpha))$$

Where the smoothing factor α is set dynamically to **0.35** to balance recent match context against foundational team capability.

B. Live Match State Dimensions

- **Required Run Rate (RRR):** Evaluated ball-by-ball as $RRR = (Runs\ Required) / (Balls\ Remaining / 6)$.
- **Wicket Pressure Index:** An exponential decay factor reflecting the resource depletion of the batting team.
- **Venue Leverage:** Historical winning percentages under lights relative to toss choices and dew point indices.

5. Dynamic Monte Carlo Simulation Engine

The defining upgrade of CricSim-X is its stochastic simulation layer. Rather than assuming linear performance, the engine forks any live game instance into $N = 10,000$ parallel paths using a Markov Chain approach.

```
# Example snippet from src/simulator.py
import numpy as np

def simulate_match_chase(target, current_score, wickets_lost, balls_left, model,
venue_features):
    win_count = 0
    simulations = 10000

    for _ in range(simulations):
        temp_score = current_score
        temp_wickets = wickets_lost
        temp_balls = balls_left

        while temp_balls > 0 and temp_wickets < 10 and temp_score <= target:
            # Vector structure mapping to current state
            features = np.array([[temp_score, temp_wickets, temp_balls,
*venue_features]])
            prob_win = model.predict_proba(features)[0][1]

            # Stochastic state adjustment
            if np.random.rand() < prob_win:
                temp_score += np.random.choice([1, 2, 4, 6], p=[0.4, 0.1, 0.3, 0.2])
            else:
                temp_wickets += 1
                temp_balls -= 1

            if temp_score > target:
                win_count += 1

    return (win_count / simulations) * 100
```

System Edge Case Handling

By integrating live state vectors, the simulation explains the paradox where a team like **RCB** retains a high global tournament win index due to explosive individual metrics, but shows a lower win floor when evaluated match-by-match on tricky away pitches.

6. Operational Commands & User Guide

The pipeline provides a clean CLI interface to walk through processing, training, and simulation steps:

Initialize Project Environment & SQLite DB Cache

```
python main.py --mode setup
```

Execute Feature Extraction & Model Stacking

```
python main.py --mode train
```

Run Predictive Simulations Across the IPL 2026 Schedule

```
python main.py --mode predict
```

Orchestrate Entire Execution Pipeline Sequentially

```
python main.py --mode all
```